
UT2 - Introducción a CSS

— Diseño de Interfaces Web &
Desarrollo de Interfaces —

Objetivos



Comienza la UT2.

En ella veremos algunos de los criterios de evaluación más relevantes del RA2:

- A. Se han reconocido las posibilidades de modificar las etiquetas HTML.
- B. Se han definido estilos de forma directa.
- C. Se han definido y asociado estilos globales en hojas externas.
- D. Se han definido hojas de estilos alternativas.

Índice

1.- Introducción

2.- Estilos

3.- La herencia

4.- Especificidad

5.- Cascada

6.- Variables

7.- Selectores

1.- Introducción

Aunque todos vosotros ya tenéis nociones previas de CSS, vamos a hacer un brevísimo repaso antes de meternos de lleno en esta parte del temario.

1.- Introducción

CSS se llama así por sus siglas en inglés: Cascading Style Sheets u Hojas de Estilo en Cascada.

Parten de un concepto simple pero muy potente: aplicar estilos (colores, formas, márgenes, etc...) a uno o varios documentos (generalmente documentos HTML, páginas webs) de forma automática y masiva.

Se denomina “en cascada” porque CSS utiliza un conjunto de reglas para decidir qué estilos se aplican cuando varias declaraciones afectan al mismo elemento. Entre otros factores, tiene en cuenta la especificidad y el orden de las reglas.

1.- Introducción

Con el CSS, perseguimos dos objetivos principales:



1. Estructura y Disposición

Diseñar y crear la estructura visual de la página:

- Afecta a la disposición de los bloques de contenido HTML.
- Crea un sistema de filas y columnas.
- Organiza componentes: barra de navegación, encabezado, pie, zona de contenidos, etc.



2. Formato y Estilo

Personalizar el aspecto de los elementos:

- Formatear y dar estilo a los textos.
- Aplicar paletas de color e identidades visuales.
- Gestionar elementos multimedia.
- Añadir animaciones y transiciones interactivas.

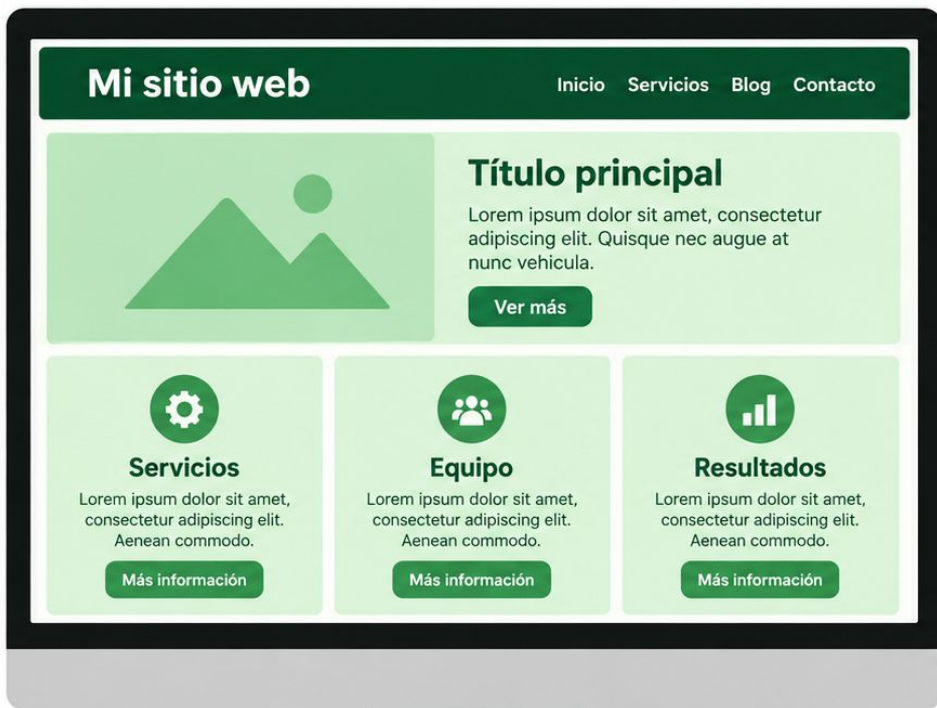
1.- Introducción

Generalmente, la estructura o información dividida en columnas, va a ver diferente según el dispositivo que lea la página, móvil , tablet, pc,

Como la imagen que se ve a continuación:

ORDENADOR → TABLET → MÓVIL

3 columnas → 2 columnas → 1 columna



ORDENADOR

3 columnas



TABLET

2 columnas



MÓVIL

1 columna

1.- Introducción

CSS2 y CSS3

CSS ha ido evolucionando con el tiempo. CSS2 consolidó muchas de las bases que seguimos utilizando: posicionamiento, tipos de display, márgenes, rellenos, bordes, etc.

Posteriormente, CSS3 incorporó y amplió numerosas características. Entre ellas encontramos propiedades y mecanismos que facilitan mucho la creación de diseños web modernos.

2.- Estilos

Los estilos son un conjunto de reglas que definen el aspecto visual y la presentación de los elementos de un documento HTML.

A continuación se explicarán los siguientes:

- Estilos en línea (Inline)
- Estilos internos o embebidos (Internal)
- Estilos externos (External)

2.- Estilos

Estilos en línea (Inline):

Se escriben directamente dentro de la etiqueta HTML utilizando el atributo style.

De esta forma ese estilo únicamente es aplicable a la etiqueta en la que se ha definido. NO APLICAR ASÍ UN ESTILO .

2.- Estilos

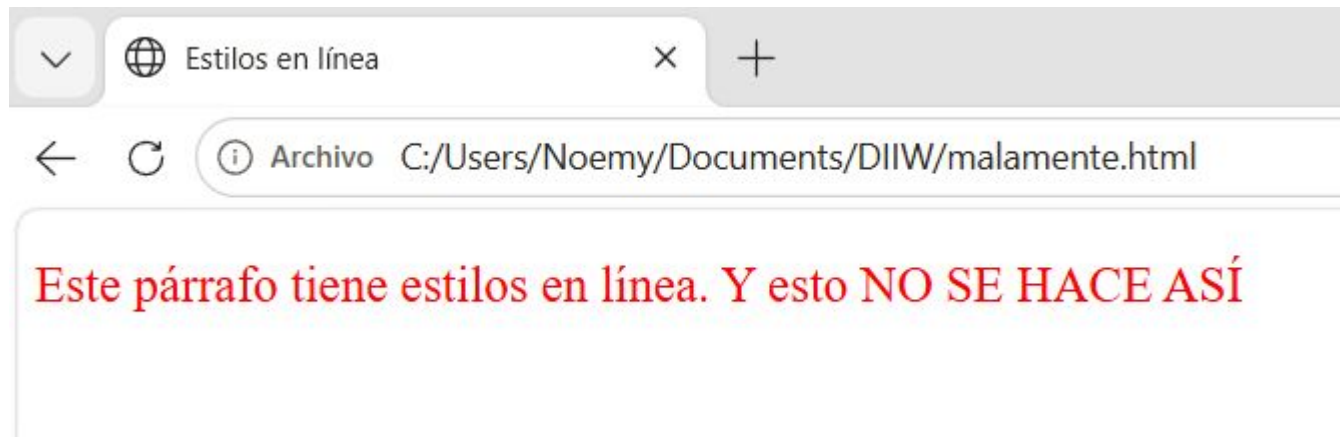
Estilos en línea

```
<> malamente.html > ...
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4   <meta charset="UTF-8">
5   <title>Estilos en línea</title>
6
7  <p style="color:  red; font-size: 20px;">
8   Este párrafo tiene estilos en línea. Y esto NO SE HACE ASÍ</p>
9
10 </head>
11
12
13 </html>
```

2.- Estilos

Estilos en línea:

Aunque a simple vista no se aprecian diferencias, si abrimos el código podemos ver que esto no está correctamente realizado.



2.- Estilos

Estilos internos o embebidos (Internal):

Se incluyen dentro de la misma página HTML, dentro de una etiqueta `<style>` situada normalmente en el `<head>`.

Ejemplo:

```
<style>
```

```
    /*Aquí las reglas de diseño*/
```

```
</style>
```

De esta forma los estilos sólo son aplicables únicamente a este archivo HTML.

2.- Estilos

```
<> embebido.html > html
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8">
6      <title>Estilos internos</title>
7
8      <style>
9          h1 {
10              color: blue;
11          }
12
13          p {
14              color: red;
15              font-size: 20px;
16          }
17      </style>
18  </head>
19
20  <body>
21
22      <h1>Soy el Dios supremo de CSS</h1>
23
24      <p>Estoy aprendiendo CSS.</p>
25      <p>Este estilo es embebido</p>
26
27  </body>
28
29  </html>
```

2.- Estilos

Esta captura es la consecución del código anterior (embebido)



2.- Estilos

Estilos externos (External):

Se declaran en un archivo separado con extensión .css, sólo contendrá las reglas CSS deseadas y se vinculan al HTML mediante la etiqueta <link>. Es la buena práctica recomendada.

```
<link href=estilos.css rel=stylesheet/>
```

2.- Estilos

Estilos externos (External):

Por un lado se genera el fichero .css (en nuestro caso se llama estilos.css)

```
# estilos.css > p
1  h1 {
2    color: blue;
3  }
4
5  p {
6    color: red;
7    font-size: 20px;
8  }
```

Y por
otro
lado, se
genera el
fichero
.html

```
<> external.html > html
1  <!DOCTYPE html>
2  <html lang="es">
3
4  <head>
5      <meta charset="UTF-8">
6      <title>Estilos externos</title>
7
8      <link rel="stylesheet" href="estilos.css">
9  </head>
10
11 <body>
12
13     <h1>Soy el Dios supremo de CSS</h1>
14
15     <p>Estoy aprendiendo CSS.</p>
16     <p>Los estilos están en otro archivo que se llama estilos.css.</p>
17
18 </body>
19
20 </html>
```

2.- Estilos

Nuevamente no se observa en la salida, pero si abrimos los ficheros vemos la realidad de los mismos.



3.- La herencia

Concepto Clave

¿Qué es la herencia?

En la aplicación de los estilos CSS, la noción de herencia interviene al tener elementos anidados.

La regla de la herencia indica que los elementos **hijos** pueden heredar las propiedades CSS de su elemento **padre**.

Ejemplo Práctico

Si aplicamos un color a un párrafo `<p>` y dentro hay una etiqueta `<strong>`, esta adoptará el mismo color.

En este caso, `<strong>` ha heredado el estilo de `<p>`.

Propiedades Heredables

Algunas de las principales propiedades CSS que se heredan:

- color, font
- text-align, direction
- letter-spacing, word-spacing
- line-height, white-space
- visibility, quotes
- border-collapse
- list-style-image

3.- La herencia

En [esta tabla](#) tenéis qué propiedades se heredan y cuáles no.

Y una [pequeña ayuda](#) a la hora de solucionar conflictos de herencia



3.- La herencia

Imagínate que un papá le pone una camiseta roja a su hijo.

¿De qué color es la ropa del hijo? ¡Roja!

¿El hijo tuvo que ir a la tienda a comprar su propia camiseta? No, simplemente usó la que le puso su papá.

En CSS, los elementos de la página web son como familias (Padres e Hijos). Si tú le dices al Padre (por ejemplo, a toda la página o a un bloque): "Quiero que tu texto sea de color azul y la letra sea grande", sus Hijos (los párrafos y textos de adentro) heredan esa orden automáticamente. No tienes que decírselo uno por uno.

3.- La herencia

Diferencias
con otros
lenguajes de
programación:

Concepto	Herencia en CSS (Diseño)	Herencia en Programación / POO (Lógica)
 ¿Entre quiénes pasa?	Entre elementos HTML reales que están unos dentro de otros en la pantalla (Padre contiene a Hijo). <pre><div> (padre) <p> (hijo)</pre>	Entre clases y código. Una clase hereda de otra clase. <pre>class Animal { ... } class Perro extends Animal { ... }</pre>
 ¿Qué se hereda?	Valores visuales (ej: color de texto, tipo de fuente, tamaño, etc.). <pre>div { color: green; }</pre>	Comportamientos y datos (ej: métodos, funciones, variables). <pre>class Animal { void comer() { ... } }</pre>
 ¿Cómo ocurre?	Es automática y visual por la estructura del HTML. <pre><div> <p></pre> El <p> hereda los estilos del <div>	Es explícita mediante código (usando palabras como <u>extends</u> o <u>inherits</u>). <pre>class Animal { ... } class Perro extends Animal { ... }</pre>

3.- La herencia

Todas las propiedades aceptan estos valores:

- **initial:** establece que el valor de la propiedad que se aplica a un elemento seleccionado tenga el mismo valor que esté establecido para esa propiedad en la hoja de estilo por defecto del navegador.
- **inherit:** establece que el valor de la propiedad que se aplica a un elemento determinado sea exactamente igual al del elemento padre. Se hereda.
- **unset:** Restablece la propiedad a su valor natural, lo que significa que si la propiedad se hereda de forma natural, actúa como inherit, y en caso contrario como initial.

4.- Especificidad

La especificidad es la manera mediante la cual los navegadores deciden qué valores de una propiedad CSS son más relevantes para un elemento y, por lo tanto, serán aplicados. La especificidad está basada en las reglas de coincidencia que están compuestas por diferentes tipos de selectores CSS.

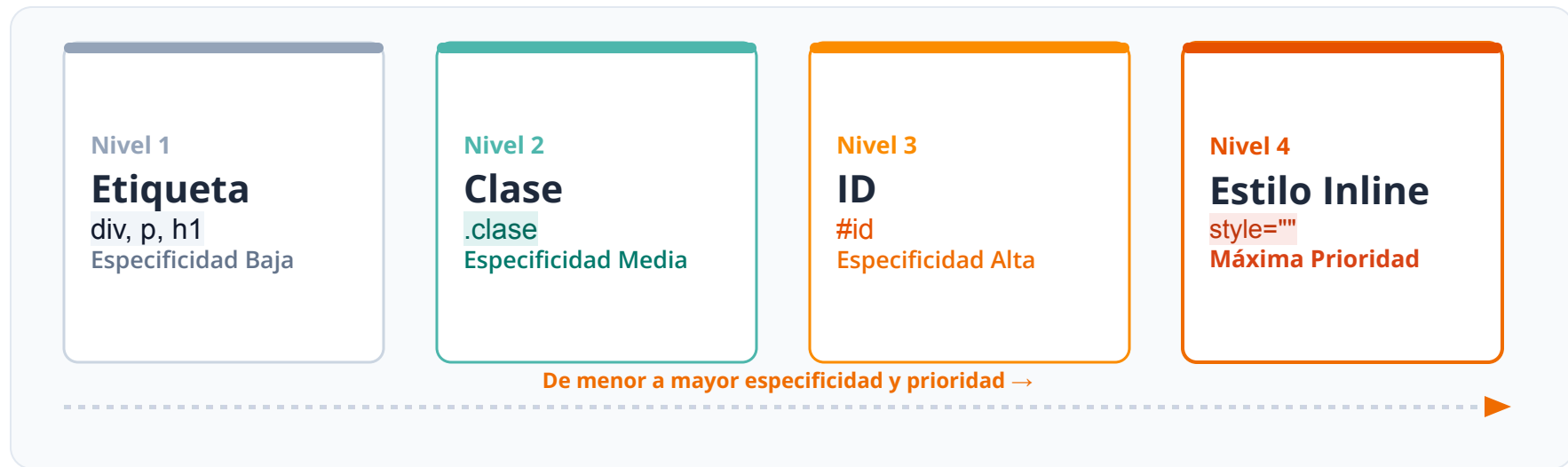
4.- Especificidad

La especificidad es un peso (importancia o valor) que se le asigna a una declaración CSS dada, determinada por el número correspondiente de cada tipo de selector. Cuando varias declaraciones tienen igual especificidad, se aplicará al elemento la última declaración encontrada en el CSS. La especificidad sólo se aplica cuando el mismo elemento es objetivo de múltiples declaraciones.

Según las reglas de CSS, en caso de que un elemento sea objeto de una declaración directa, ésta siempre tendrá preferencia sobre las reglas heredadas de su ancestro.

4.- Especificidad

Regla principal: Cuanto más específico es un selector, mayor prioridad tiene en la aplicación de estilos.



4.- Especificidad

Imaginaos este fragmento de código:

Html:

```
<p id="especial" class="importante">Hola mundo</p>
```

CSS:



```
p {  
    color: blue;  
}  
.importante {  
    color: green;  
}  
#especial {  
    color: red;  
}
```

4.- Especificidad

Las tres reglas intentan cambiar el color del mismo párrafo. Entonces CSS necesita decidir cuál gana.

Aquí entra la especificidad:

Etiqueta

p

Menos específico

Prioridad baja en el cálculo de estilos

Clase

.importante

Más específico

Sobrescribe las reglas de etiqueta

ID; Gana!

#especial

Todavía más específico

Máxima prioridad de las tres

Mayor especificidad y prioridad →

5.- Cascada

En un primer nivel de simplicidad, la cascada en las hojas de estilo significa que el orden de las reglas importa en CSS: cuando dos reglas tienen la misma especificidad, se aplica la que aparece en último lugar en el CSS.

```
h1{color:red}
```

```
h1{color:blue}
```

El color de h1 será azul.

```
<> cascada.html > html
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <title>Ejemplo de cascada CSS</title>
6
7      <style>
8          h1 {
9              color: red;
10         }
11
12         h1 {
13             color: blue;
14         }
15     </style>
16 </head>
17
18 <body>
19
20     <h1>Soy el Dios supremo de CSS</h1>
21
22 </body>
23 </html>
```

5.- Cascada



6.- Variables

En CSS, las propiedades personalizadas (también conocidas como variables) son entidades definidas por autores de CSS que contienen valores específicos que se pueden volver a utilizar en un documento. El valor de la variables puede ser nombres de fuentes, medidas de cualquier tipo, colores, incluso valores calculados.

6.- Variables

El uso de variables no estaba presente en las primeras versiones de CSS.

Aunque aquí nos referimos a ellas como variables, la forma oficial de llamarlas es custom properties (propiedades personalizadas).

6.- Variables

Se establecen mediante la notación de propiedades personalizadas por ejemplo:

```
--main-color: black;
```

y se acceden mediante la función `var()` :

```
color: var(--main-color);
```

6.- Variables

Sitios web complejos tienen cantidades muy grandes de CSS, a menudo con una gran cantidad de valores repetidos. Por ejemplo, el mismo color se puede utilizar en cientos de lugares diferentes, lo cual requiere buscar y reemplazar globalmente si necesitamos cambiar ese color.

Las variables CSS permiten que un valor se almacene en un lugar y luego se haga referencia en varios otros lugares. Un beneficio adicional son los identificadores semánticos.

6.- Variables

Por ejemplo `--main-text-color` es más fácil de entender que `#00ff00`, especialmente si este mismo color también se utiliza en otro contexto.

Las variables CSS están sujetas a la cascada, y heredan su valor de su padre.

6.- Variables

Uso básico

```
elemento {  
  --main-bg-color: brown;  
}
```

Ten en cuenta que el selector que usemos para las reglas de estilo define el ámbito (scope) en el que podremos usar la propiedad personalizada (variable). Una buena práctica común es declarar variables en la pseudo-clase :root, cuyo ámbito es el propio HTML:

```
:root {  
  --main-bg-color: brown;  
}
```

6.- Variables

Para acceder al valor de la variable:

```
elemento {
```

```
    background-color: var(--main-bg-color);
```

```
}
```

7.- Selectores

A grandes rasgos, se puede decir que los selectores son la parte de una regla CSS que indica qué elemento o elementos HTML queremos seleccionar para aplicarles un estilo.

[Aquí](#) y [aquí](#) tenéis información pormenorizada de los mismos.

7.- Selectores

Para crear diseños web profesionales, es imprescindible conocer y dominar los selectores de CSS.

Una regla de CSS está formada por una parte llamada "selector" y otra parte llamada "declaración".

La declaración indica "qué hay que hacer" y el selector indica "a quién hay que aplicarlo". Por lo tanto, los selectores son imprescindibles para aplicar de forma correcta los estilos CSS en una página.

7.- Selectores

A un mismo elemento HTML se le pueden aplicar varias reglas CSS y cada regla CSS puede aplicarse a un número ilimitado de elementos. En otras palabras, una misma regla puede aplicarse sobre varios selectores y un mismo selector se puede utilizar en varias reglas.

Una regla CSS es el conjunto formado por un selector y un bloque de declaraciones que indica qué elementos HTML queremos modificar y qué estilos queremos aplicarles.

7.- Selectores

Aquí tenemos:

- Selector: p → indica a quien se aplica el estilo.
- Declaración: color: red; indica qué estilo aplicar.
- Declaración: font-size: 18px;
- Regla CSS: todo el conjunto.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="UTF-8">
5    <title>Ejemplo de regla CSS</title>
6
7    <style>
8      p {
9        color: ■ red;
10       font-size: 18px;
11     }
12   </style>
13 </head>
14
15 <body>
16
17   <h1>Mi primera regla CSS</h1>
18
19   <p>Hola Holita.</p>
20
21   <p>Alumnito.</p>
22
23 </body>
24 </html>
```

7.- Selectores

La siguiente lista de tipos de selectores incrementa en función de la especificidad:

- Selectores de tipo y pseudo-elementos
- Selectores de clase, selectores de atributos y pseudo-clases
- Selectores de ID

Bibliografía

<https://lenguajecss.com/css/introduccion/que-es-css/>

<https://developer.mozilla.org/es/docs/Web/CSS/Guides/Cascade/Specificity>

<https://developer.mozilla.org/>

Algunas de las imágenes de este documento se han creado a través de chatgpt.com